

PyGameMaker — Tutorial 4: Breakout — Teacher Guide

Companion for the in-app tutorial (**Help > Tutorials > Breakout: Brick Breaker**, 7 pages) and the matching student handout and worksheet. Students should have finished Tutorials 2 and 3 (events, collisions, score, draw, solid objects).

Overview

Students build Breakout in four phases and test after each: a bouncing ball, one row of bricks, four rows, and a game controller with lives. New ideas: **parent and child objects** (one collision rule for many brick types), a **death zone** instead of a bottom wall, **lives**, and a **Game Over with highscore**.

The key idea is inheritance: "a brick red *is a* brick, so it gets whatever a brick gets". Ask students to predict what happens when the parent is not set on the new colour.

Suggested Timing (60-90 minutes)

The tutorial says 25-30 minutes for a confident student. It has seven pages and many objects, so plan a double period or two sessions.

Segment	Time	What happens
Recap and predict	5 min	Show the finished game; ask "which objects do we need?"
Sprites (page 2)	10 min	Let students import the sample images from Tutorials/04_breakout/assets/ to save time
Paddle, ball and room (pages 3-4)	20 min	Phase 1; everyone plays a bouncing ball
Bricks (pages 5-6)	20 min	Phases 2-3; the parent object
Controller, lives, Game Over (page 7)	15 min	Phase 4
Worksheet / exit ticket	5-10 min	Parts A-C

For a 45-minute slot, do sprites and Phase 1 in the first session and the rest in a second.

Step-by-Step Walkthrough and Common Issues

Phase 1: bouncing ball

- Two wall objects (side and top) exist so the ball can reverse the right direction: side wall reverses horizontal, top wall and paddle reverse vertical.
- **Solid** on walls and paddle is needed; the ball is **not** Solid. Every bounce is a separate Collision event on the ball.
- The paddle needs its own **Collision with obj_wall_side** event with **Stop movement**; otherwise it goes through the side walls.

- **There is no bottom wall on purpose.** The invisible death zone at the bottom catches the ball and jumps it to (320, 100). Students often place a wall at the bottom; ask "how would the ball ever get lost?".
- The ball keeps bouncing forever between walls and the paddle; that is **expected**. The paddle is only needed to stop the ball from falling.

Phase 2-3: bricks and the parent object

- obj_brick_parent has no sprite and is Solid. Each colour needs **Parent = obj_brick_parent** and **Solid**. Forgetting the parent on a new colour is the classic mistake: that colour is a wall that never breaks.
- The ball's brick event is on obj_brick_parent only. Show students that adding a fifth colour needs **no** change to the ball.
- Each brick gives 10 points regardless of colour (the tutorial leaves different scores as a challenge).
- Use **Snap to Grid** (32×16) when placing bricks, or rows will be uneven.

Phase 4: game controller

- The controller must be **placed in the room**, like the controller in Tutorial 2.
- Lives are set in the controller's **Create** event; the ball subtracts a life in its death-zone event (*Add to lives -1* before the jump).
- **No More Lives** is an event that fires automatically when lives reach zero. The tutorial asks for *Show message*, *Show Highscore*, *End Game* in that order and warns that order matters. On the desktop player the highscore is shown either way, but keep the tutorial's order: it is the safe one.
- The score and lives are drawn in white at y 10, over the grey top wall row; they stay readable.

Reference projects. Download the checkpoint projects from the wiki (04_breakout_checkpoints.zip): one project for the end of each phase. Give a stuck student the previous phase so they can continue.

Vocabulary Introduced

Term	What it means here
Parent object	A template; children inherit its events
Child object	An object whose Parent field names another object
Death zone	An invisible object at the bottom that catches the ball
Lives	A counter of how many times the ball may be lost
Highscore table	A saved list of the best scores

Discussion Questions

- Why write one collision event for the parent instead of one per colour?
- Why does the ball reverse *vertical* movement on a brick?
- What could we change so the game is harder or fairer? (Speed, paddle size, angle of bounce.)
- Which parts of the game are events, and which are actions?

Differentiation

- **Support:** give the Phase 1 checkpoint and start at bricks; import the sample sprites instead of drawing.
- **Extension:** different points per colour; ball speeds up; a second level with another brick layout; a sound at the end.

Worksheet Answer Key

Part A: 1-C, 2-A, 3-B, 4-D. **Part B:** 1. Reverse horizontal movement. 2. Destroy other instance. 3. Add to lives -1 (set lives, relative -1). 4. Show message. 5. End Game. **Part C:**

- 1 Set its **Parent** to obj_brick_parent (and Solid). No, the ball's event for the parent already covers it.
- 2 Up and to the right. On the screen y grows downward, so a negative vertical speed means up.
- 3 Show Highscore must come before End Game because the tutorial says ending the game first can stop the highscore from appearing. (In the desktop player both orders work; the safe order is still best.)

Parts D and E: completion and reflection.

Rubric: Breakout

Level	What the game shows
4 - Complete	Paddle and ball work; bricks of all colours break through one parent event and give points; lives and score are displayed; Game Over, highscore and end work
3 - Working	Paddle, ball and bricks work; lives or Game Over is missing
2 - Partly there	Paddle and ball work; bricks do not break, or only one colour breaks
1 - Started	Objects exist, but the ball does not bounce or the paddle does not move

End-of-Session Checklist

- ☐ Every student has a saved project (Ctrl+S)
- ☐ Every student reached at least Phase 2 (a brick can be broken)
- ☐ Note who needs the checkpoint project next time